

Challenge Write-Up Template

Challenge Information

Student Name	Pascua, Diana KC S.
Section / Class	BSIT-3G
Challenge Name	Password Profiler
Category	General Skills
Points / Difficulty	Easy
Date Solved	8/16/26
Flag Format Given	picoCTF{...}

1. Challenge Description

This challenge gave me three files in my home directory: `userinfo`, containing personal details about a fictional "victim" (Alice Johnson); `hash`, containing the SHA-1 hash of Alice's real password; and `check_password`, a script meant to test candidate passwords against that hash. The goal was to use Alice's personal information to build a smart, targeted wordlist of likely passwords — the same technique real attackers use for social-engineering-based password guessing — and then find the one candidate whose hash actually matched.

2. Initial Analysis / Reconnaissance

Since the challenge page named three specific files up front, I didn't need to poke around blindly — I went straight to find `~ -type f \( -name "userinfo*" -o -name "hash*" \)` and a matching find for `check_password*` to confirm exactly where each one lived and read their contents. `userinfo` turned out to be exactly what it sounds like: name, nickname, birthdate, and family details for Alice. `hash` held a single SHA-1 digest. That combination — personal data plus a target hash, no source code to reverse and no service to attack — told me this wasn't a binary-exploitation or web challenge; it was a password-profiling / dictionary-attack challenge, and the hint pointing at CUPP (Common User Passwords Profiler) confirmed it.

3. Tools and Commands Used

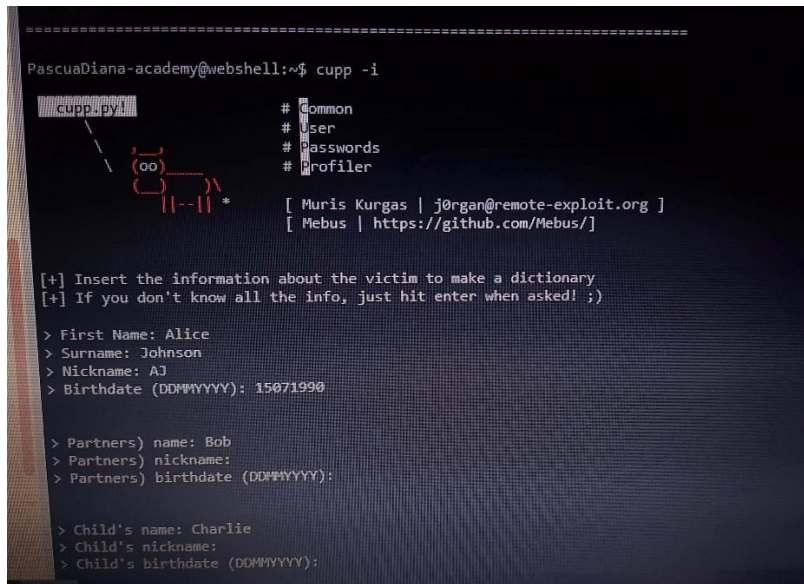
Tool / Command	What it was used for	Result / Output
<code>find ~ -name "userinfo*" / "hash*" / "check_password"</code>	Find the challenge's files.	Located userinfo, hash, and check_password.
<code>cupp -i</code>	Build a wordlist from Alice's personal info.	Answered the prompts with her name, nickname, and birthdate; saved 5,180 guesses to alice.txt.
<code>ls -l alice.txt</code>	Confirm the wordlist actually got created.	File existed, 52,572 bytes.
<code>while read p; do ...sha1sum ... done < alice.txt</code>	Hash every guess and check it against the target hash.	One match: Aj_15901990.

4. Solution Walkthrough

Step 1

Read the victim's personal details

I located and read `userinfo`, which laid out Alice Johnson's identifying details: first name Alice, surname Johnson, nickname AJ, birthdate 15071990, partner named Bob, and a child named Charlie. That's precisely the kind of information people tend to build passwords out of — names, nicknames, and dates that matter to them — which is why it was worth feeding into a dedicated password-profiling tool rather than guessing by hand.



```
=====
PascuaDiana-academy@webshell:~$ cupp -i

cupp.py!
# Common
# User
# Passwords
# Profiler

[ Muris Kurgas | j0rgan@remote-exploit.org ]
[ Mebus | https://github.com/Mebus/ ]

[+] Insert the information about the victim to make a dictionary
[+] If you don't know all the info, just hit enter when asked! ;)

> First Name: Alice
> Surname: Johnson
> Nickname: AJ
> Birthdate (DDMMYYYY): 15071990

> Partners) name: Bob
> Partners) nickname:
> Partners) birthdate (DDMMYYYY):

> Child's name: Charlie
> Child's nickname:
> Child's birthdate (DDMMYYYY):
```

Step 2

Generate a targeted wordlist with CUPP

I ran `cupp -i` to launch CUPP's interactive profiler. It walks through a series of prompts — first name, surname, nickname, birthdate, partner's info, child's info, pet, company — and for anything I didn't have from `userinfo`, I just hit enter to skip it, exactly as CUPP's own prompt suggested ("If you don't know all the info, just hit enter when asked!"). I answered no to adding key words, special characters, trailing numbers, or leet-speak substitutions, since `userinfo` didn't suggest Alice used any of those habits. CUPP then combined and permuted everything I gave it — different capitalizations, name/date orderings, and so on — into a wordlist of 5,180 candidates saved to `alice.txt`.

Step 3

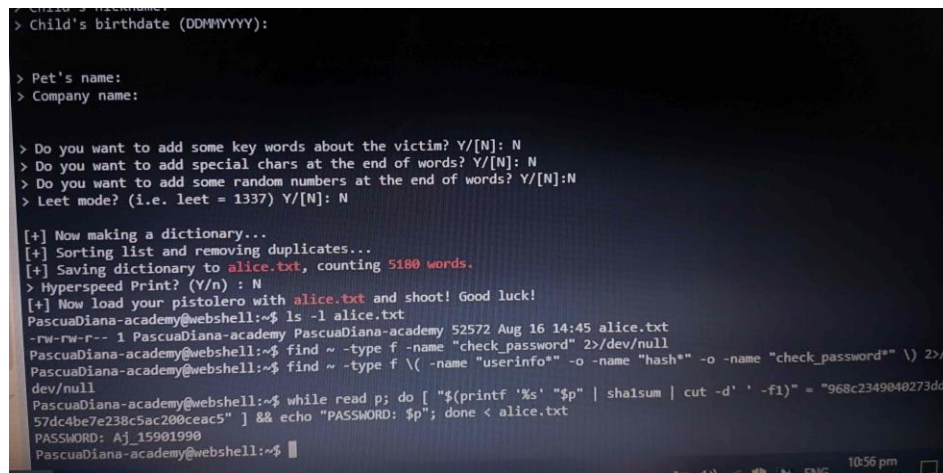
Verify the wordlist and read the target hash

I ran `ls -l alice.txt` to make sure the file had actually been written (52,572 bytes — reassuring, since an empty or missing wordlist would mean I'd answered the CUPP prompts incorrectly), and confirmed the exact SHA-1 value from hash: 968c234040273dd57dc4be7e238c5ac200ceac5.

Step 4

Hash every candidate and look for a match

With no ready-made cracking tool like John or hashcat installed on the box, I wrote a small shell loop to do the same job by hand: `while read p; do [ "$(printf '%s' "$p" | sha1sum | cut -d' ' -f1)" = "968c234040273dd57dc4be7e238c5ac200ceac5" ] && echo "PASSWORD: $p"; done < alice.txt`. This reads `alice.txt` one candidate password at a time, computes its SHA-1 digest with `sha1sum`, strips off the filename `sha1sum` appends, and compares it against the target hash — printing the candidate only when the digests match. It printed exactly one line: `PASSWORD: Aj_15901990`.

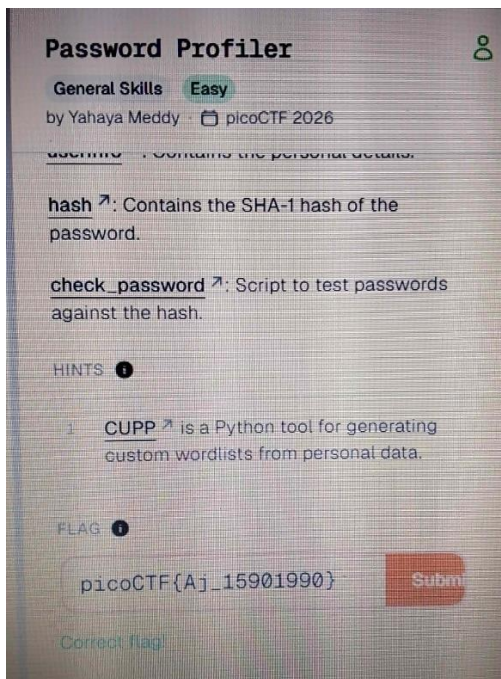


```
> Child's birthdate (DDMMYYYY):  
> Pet's name:  
> Company name:  
  
> Do you want to add some key words about the victim? Y/[N]: N  
> Do you want to add special chars at the end of words? Y/[N]: N  
> Do you want to add some random numbers at the end of words? Y/[N]: N  
> Leet mode? (i.e. leet = 1337) Y/[N]: N  
  
[+] Now making a dictionary...  
[+] Sorting list and removing duplicates...  
[+] Saving dictionary to alice.txt, counting 5180 words.  
> Hyperspeed Print? (Y/n) : N  
[+] Now load your pistolero with alice.txt and shoot! Good luck!  
PascuaDiana-academy@webshell:~$ ls -l alice.txt  
-rw-rw-r-- 1 PascuaDiana-academy PascuaDiana-academy 52572 Aug 16 14:45 alice.txt  
PascuaDiana-academy@webshell:~$ find ~ -type f -name "check_password" 2>/dev/null  
PascuaDiana-academy@webshell:~$ find ~ -type f \( -name "userinfo" -o -name "hash*" -o -name "check_password*" \) 2>/dev/null  
PascuaDiana-academy@webshell:~$ while read p; do [ "$(printf '%s' "$p" | sha1sum | cut -d' ' -f1)" = "968c234040273dd57dc4be7e238c5ac200ceac5" ] && echo "PASSWORD: $p"; done < alice.txt  
PASSWORD: Aj_15901990  
PascuaDiana-academy@webshell:~$
```

Step 5

Submit the flag

The challenge's flag format wrapped the recovered password directly: `picoCTF{Aj_15901990}`, which the challenge page confirmed as "Correct flag!"



5. Flag

picoCTF{Aj_15901990}

6. Key Concept / What I Learned

This challenge is really about targeted dictionary attacks against weak, personally-derived passwords. Hashing a password (even with a decent algorithm) protects it against casual viewing, but it does nothing to protect against an attacker who can guess the plaintext — and people who base passwords on names, nicknames, and birthdates hand attackers most of the guesswork for free. Tools like CUPP automate exactly that kind of social-engineering-informed guessing, which is why security guidance always pushes for passwords that have nothing to do with public or discoverable personal information.

7. References

CUPP (Common User Passwords Profiler) by Muris Kurgas (j0rgan) and Mebus
